

Homework 0: Julia and JuMP exercises

This assignment verifies that you can use the course's Julia environment, edit and run Jupyter notebooks, create plots and mathematics, build a JuMP model, and export saved work to PDF.

1. Complete `julia-tutorial.ipynb` first.
2. Fill in every requested Markdown or code cell in this notebook.
3. Run all cells from top to bottom with the Julia 1.12 kernel.
4. Save the notebook and confirm that every requested result is visible.
5. With this notebook active, run **ISyE 524: Export active notebook to PDF (TinyTeX)** or **ISyE 524: Export active notebook to PDF (system TeX)**, according to the LaTeX setup chosen during installation.
6. Review `submissions/hw00.pdf`, upload it to Gradescope, and match its pages to the assignment questions.

Work only in the copy under `student-work/hw00/`, not in the tracked template under `assignments/hw00/`.

Course-work acknowledgement

Read the following statements and acknowledge them by typing your name in the next Markdown cell.

1. You are encouraged to discuss homework problems with classmates and may work in groups when the course policy permits it.
2. The work you submit must be your own. Do not exchange files containing code or answers to homework questions.
3. Many assignments require Julia and JuMP. The course provides a tested environment and instructions, but you are responsible for maintaining a working installation and backing up your work.
4. Homework is submitted through Gradescope as a PDF of the completed notebook, not as notebook source code. Keep answers in assignment order and use clear question headings to make page matching easier.
5. Optimization modeling is learned by doing the modeling yourself.
6. Academic misconduct is not tolerated. Review the university's [academic misconduct information](#).

1.0 Signature

I have read and understand the course-work statements above.

Name: Dustin Pick

1.1 Working with arrays and tuples

Define the array

```
square = [1, 2, 3]
```

and the tuple

```
round_tuple = (4, 5, 6)
```

```
square = [1, 2, 3]
```

```
3-element Vector{Int64}:
```

```
1
```

```
2
```

```
3
```

```
round_tuple=(4, 5, 6)
```

```
(4, 5, 6)
```

Access the first element of the array and the tuple, and add them together.

```
sum = square[1]+round_tuple[1]
```

```
5
```

Change the first element of the array to be equal to the first element of the tuple.

```
square[1]=round_tuple[1]  
square
```

```
3-element Vector{Int64}:
```

```
4
```

```
2
```

```
3
```

Consider the assignment `round_tuple[3] = square[3]`. Why would Julia reject it? Explain below, but keep the invalid assignment commented out so **Run All** can finish successfully.

Why will this not work?

```
# round_tuple[3] = square[3] would fail because tuples are immutable and thus th
```

1.2 Dictionaries

Create a dictionary which lists three of your favorite restaurants and their ranking (1, 2 or 3).

```
resturants = Dict(  
    3 => "Taco Bell",  
    1 => "Qdoba",  
    2 => "Jimmy Johns",  
)
```

Dict{Int64, String} with 3 entries:

```
2 => "Jimmy Johns"  
3 => "Taco Bell"  
1 => "Qdoba"
```

1.3 Matrices (two-dimensional arrays)

Create the following Julia matrix: $B = \begin{bmatrix} 1 & 2 & 1 \\ 3 & 0 & 1 \\ 0 & 2 & 4 \end{bmatrix}$

(Please also look in this markdown cell about how to write matrices in your notebooks)

```
B=[  
    1 2 1;  
    3 0 1;  
    0 2 4;  
]
```

3×3 Matrix{Int64}:

```
1 2 1  
3 0 1  
0 2 4
```

Change the first element in the first row of B to 5, and check whether it is even. The expression `a % b` returns the remainder after dividing a by b.

```
B[1,1]=5
```

5

```

if B[1,1]%2 == 0
    println("B(1,1) is even")
else
    println("B(1,1) is odd")
end

```

B(1,1) is odd

1.4 For loops

Write a for loop to print the integers from 1 to 5.

```

for i in 1:5
    println(i)
end

```

1
2
3
4
5

Now write a for loop to go through every element in the above matrix B , check if it is odd. If it is, then add 1 to that element of the matrix, and print your resulting matrix.

```

SizeB=size(B) #format rows, columns
for i in 1:SizeB[1]
    for j in 1:SizeB[2]
        if B[i,j]%2 != 0
            B[i,j] +=1
        end
    end
end

```

B

```

3×3 Matrix{Int64}:
 6  2  2
 4  0  2
 0  2  4

```

1.5 List Comprehensions

Create a list (vector) of integers called `my_list` that contains the integers 1 to 10 in one line using a list comprehension

```
my_list=[i for i in 1:10]
```

```
10-element Vector{Int64}:
```

```
1
2
3
4
5
6
7
8
9
10
```

Create a list (vector) called `my_list2` of the square of all odd numbers from 13 to 17 using a list comprehension

```
my_list2=[sqrt(i) for i in 13:17 if i%2 == 1]
```

```
3-element Vector{Float64}:
```

```
3.605551275463989
3.872983346207417
4.123105625617661
```

1.6 Functions

Write a function named `my_func` that takes an integer n and returns an array containing the integers from 1 through n . Try the function with input 5.

```
function my_func(n)
    return [i for i in 1:n]
end
my_func(5)
```

```
5-element Vector{Int64}:
```

```
1
2
3
4
5
```

1.7 Plots

This exercise verifies that the course environment can create a plot and preserve it in the exported PDF. Plots is already included in the repository environment.

```
using Plots
```

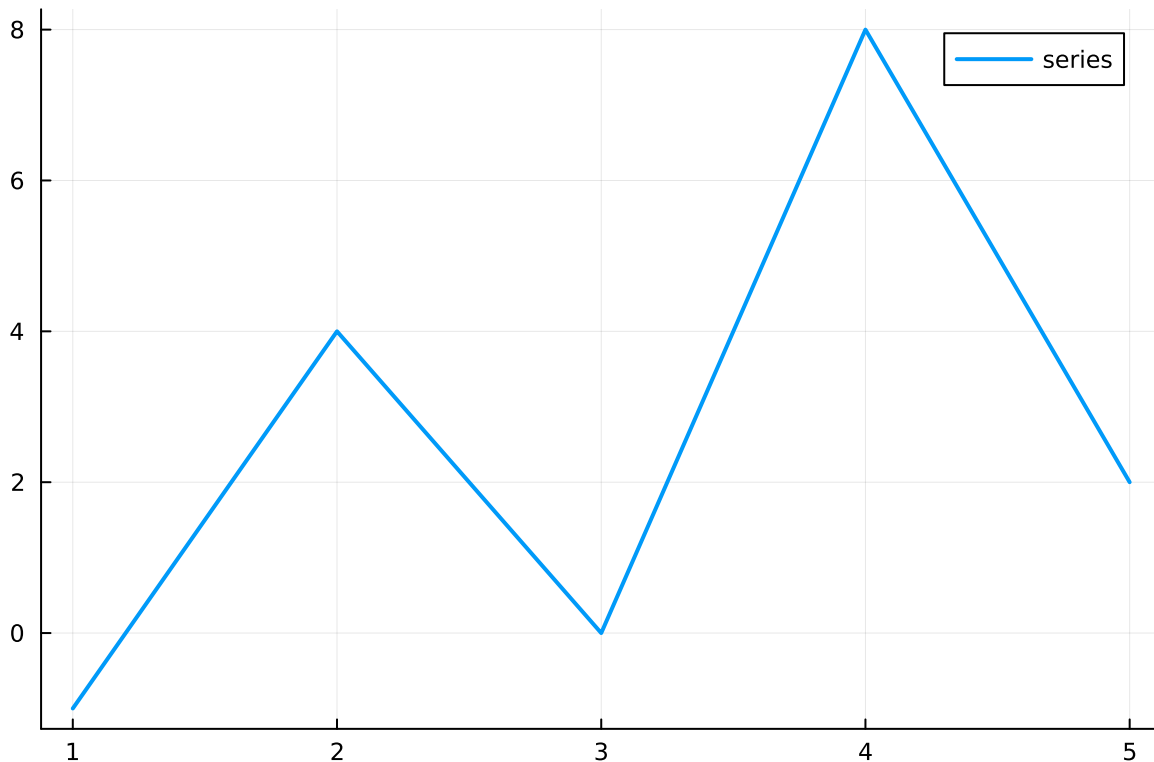
Confirm that the repository project is active. The printed path should end in `isye524-students-julia/Project.toml`.

```
Base.active_project()
```

```
"/home/pickd/M&S Masters/M-S-Masters/ISYE 542/isye524-students-julia/Project.toml"
```

Run the cell below to make a simple line graph.

```
x_plot = 1:5  
y_plot = [-1, 4, 0, 8, 2]  
plot(x_plot, y_plot; linewidth = 2, label = "series")
```



1.8 Mathematics in Markdown

This section verifies that mathematical notation renders correctly in your notebook and PDF.

Markdown can contain lists:

- Item 1
- Item 2
- Item 3

Use single dollar signs for inline mathematics, such as $x_1 = 4$, and double dollar signs for displayed mathematics:

$$\sum_{j=1}^8 \sin(x_j) \leq 14.$$

Aligned equations can be written as

$$\begin{aligned} x_1 &= 4, \\ x_2 &= 18, \\ \sum_{j=1}^n a_{ij}x_j &\geq b_i \quad \forall i \in M. \end{aligned}$$

A matrix can be written as

$$A = \begin{bmatrix} 1 & 7 & 3 \\ -1 & \alpha & \beta \end{bmatrix}.$$

Edit this cell to inspect the Markdown source.

1.9 Pictures and handwritten work

To embed an image in VS Code, edit a Markdown cell, drag a PNG or JPEG into the cell, and choose **Insert Image as Attachment**. VS Code inserts Markdown similar to:

```
![Description of the image](attachment:image.png)
```

The image is stored inside the notebook and travels with it. For several large images, place an `images/` folder beside the notebook and use a relative path such as `images/model.png`. Avoid HEIC images because support varies by platform.

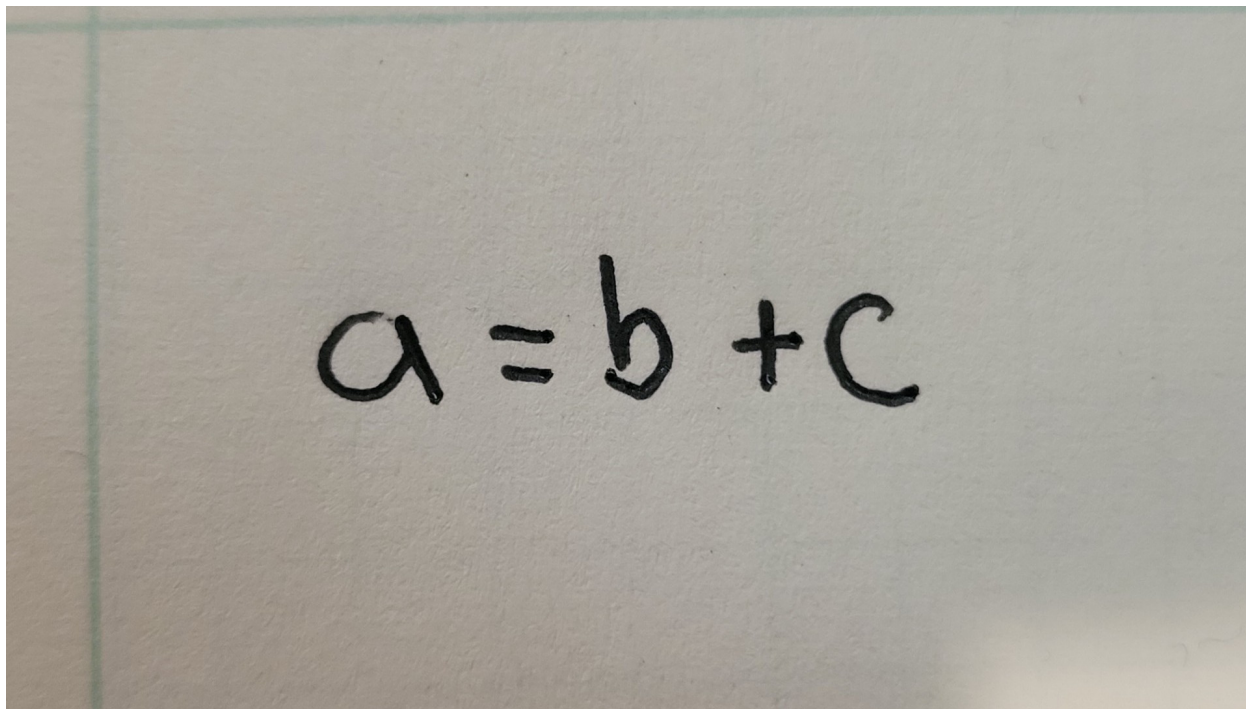


Figure 1: test equation-2.jpg

Insert a picture in this Markdown cell. If you expect to submit handwritten portions of later assignments, use this exercise to verify that your preferred image workflow appears correctly in the exported PDF.

An introduction to JuMP

This section introduces the basic JuMP modeling workflow. Complete the Julia exercises above first.

Useful references include:

- [JuMP documentation](#)
- [Julia Programming for Operations Research](#)
- [Julia Discourse optimization category](#)

Some material will be discussed during the first weeks of class. It is fine if the solver details are not yet familiar.

2.0 JuMP setup

JuMP, HiGHS, and Ipopt are already installed in the pinned course environment. Load them without running `Pkg.add` or `Pkg.update`.

```
using JuMP, HiGHS, Ipopt
import MathOptInterface as MOI
```

The first import in a Julia session may take a little longer while packages compile. A completed cell displays a normal execution count rather than `[*]`.

```
println("Active course project: ", Base.active_project())
```

```
Active course project: /home/pickd/M&S Masters/M-S-Masters/ISYE 542/isy524-
students-julia/Project.toml
```

Package versions are managed by the repository. If a required package is missing, rerun **ISyE 524: Set up / refresh Julia environment** instead of changing the environment from this notebook.

The course environment is ready.

2.1 Building an optimization model. JuMP Variables

First, load the JuMP package into your current environment.

```
using JuMP
```

Now you can start building your optimization model, which we will also refer to as a **JuMP model**!

Remember that there are three components to every optimization problem:

1. Decision variables (the values we are allowed to determine)
2. Objective (the goal we want to achieve, which is expressed as a function of the decision variables)
3. Constraints (limitations that describe which choices are possible to make, also expressed as functions of the decision variables)

We will go through how to model these three parts using Julia and JuMP one by one. Let's start with defining decision variables:

```
first_model = Model()
@variable(first_model, y >= 0)
@variable(first_model, 1 <= z <= 2)
first_model
```

```
A JuMP Model
 solver: none
 objective_sense: FEASIBILITY_SENSE
 num_variables: 2
 num_constraints: 3
```

```

    VariableRef in MOI.GreaterThan{Float64}: 2
    VariableRef in MOI.LessThan{Float64}: 1
Names registered in the model
: y, : z

```

```

# Let's find the lower bound of the z variable
JuMP.lower_bound(z)

```

1.0

Other ways to create variables

Sometimes, we need to create problems with MANY variables. Then it is useful to not have to create each variable separately.

A useful feature is that we can create arrays of JuMP variables.

```

model = Model()
@variable(model, x[1:4] >= 0)
x

```

```

4-element Vector{VariableRef}:
 x[1]
 x[2]
 x[3]
 x[4]

```

The indices of the arrays don't have to be integers. They can be anything, like a string "name" or a symbol :symbol_name.

Let's create a variable that uses both number indices and symbols!

```

model = Model()
@variable(model, x[i = 1:2, j = [:A, :B]] >= i)

println("Printing my optimization variable: ")
println()
println(x)
println()
println("The lower bound of the first element is ", JuMP.lower_bound(x[1, :A]))

```

Printing my optimization variable:

```

2-dimensional DenseAxisArray{VariableRef,2,...} with index sets:
  Dimension 1, Base.OneTo(2)
  Dimension 2, [:A, :B]
And data, a 2x2 Matrix{VariableRef}:

```

```
x[1,A]  x[1,B]
x[2,A]  x[2,B]
```

The lower bound of the first element is 1.0

Another example, with strings as names:

```
model = Model()
@variable(model, x[i = 1:4, j = ["one", "two"]] >= i)
x
```

2-dimensional DenseAxisArray{VariableRef,2,...} with index sets:

```
Dimension 1, Base.OneTo(4)
Dimension 2, ["one", "two"]
```

And data, a 4×2 Matrix{VariableRef}:

```
x[1,one]  x[1,two]
x[2,one]  x[2,two]
x[3,one]  x[3,two]
x[4,one]  x[4,two]
```

JuMP variable names must be unique within a model. Use distinct names when the variables represent distinct decisions.

```
model = Model()
@variable(model, x_one >= 1)
@variable(model, x_two >= 2)
model
```

A JuMP Model

```
solver: none
objective_sense: FEASIBILITY_SENSE
num_variables: 2
num_constraints: 2
  VariableRef in MOI.GreaterThan{Float64}: 2
Names registered in the model
  :x_one, :x_two
```

Binary and integer variables

By default, the decision variables in Julia are continuous variables. However, we can also create binary and integer variables as follows:

```
model = Model()
@variable(model, x >= 1, Int)
@variable(model, y, Bin)
model
```

```

A JuMP Model
solver: none
objective_sense: FEASIBILITY_SENSE
num_variables: 2
num_constraints: 3
  VariableRef in MOI.GreaterThan{Float64}: 1
  VariableRef in MOI.Integer: 1
  VariableRef in MOI.ZeroOne: 1
Names registered in the model
  :x, :y

```

2.2 JuMP Constraints

Now that we've seen how to create variables, let's look at **constraints**.

Remember that constraints are limitations on the valid choices of decision variables (for example, the production of football and soccer trophies is limited by the available amount of wood). They may involve one or more decision variable, and be formulated as inequalities or equalities.

Let's formulate the following constraints with decision variables $x \geq 0$ and $y \geq 0$:

$$2x + y \leq 1$$

$$2x + y \geq 1$$

$$2x + y = 1$$

Here is an example:

```

model = Model()
@variable(model, x >= 0)
@variable(model, y >= 0)

@constraint(model, c_less_than, 2x + y <= 1)
@constraint(model, c_greater_than, 2x + y >= 1)
@constraint(model, c_equal_to, 2x + y == 1)

print(model)

```

```

Feasibility
Subject to
  c_equal_to : 2 x + y = 1
  c_greater_than : 2 x + y ≥ 1
  c_less_than : 2 x + y ≤ 1
  x ≥ 0
  y ≥ 0

```

Similar to the optimization variables, we can access the constraints using their names:

```
print(c_equal_to)
```

c_equal_to : $2x + y = 1$

2.3 JuMP Objective Functions

Now let's look at the last main part of the optimization model: The objective function.

Note two important aspects: 1. The objective is formulated as a function of the optimization problem. 2. We need to specify whether we want to maximize or minimize this function.

Minimization problem (i.e., minimizing the objective function):

```
model = Model()
@variable(model, x >= 0)

@objective(model, Min, 2x + 1)

model
```

```
A JuMP Model
 solver: none
 objective_sense: MIN_SENSE
  objective_function_type: AffExpr
 num_variables: 1
 num_constraints: 1
  VariableRef in MOI.GreaterThan{Float64}: 1
 Names registered in the model
  :x
```

Maximization problem (i.e., maximizing the objective function):

```
model = Model()
@variable(model, x <= 2)

@objective(model, Max, 2x + 1)

model
```

```
A JuMP Model
 solver: none
 objective_sense: MAX_SENSE
  objective_function_type: AffExpr
 num_variables: 1
 num_constraints: 1
  VariableRef in MOI.LessThan{Float64}: 1
 Names registered in the model
  :x
```

2.4 DO IT YOURSELF!

Try to build the optimization model.

Top Brass Trophy Company makes large championship trophies for youth athletic leagues. At the moment, they are planning production for fall sports: football and soccer. Each football trophy has a wood base, an engraved plaque, a large brass football on top, and returns 12 dollars in profit. Soccer trophies are similar except that a brass soccer ball is on top, and the unit profit is only 9 dollars. Since the football has an asymmetric shape, its base requires 4 board feet of wood; the soccer base requires only 2 board feet. At the moment there are 1000 brass footballs in stock, 1500 soccer balls, 1750 plaques, and 4800 board feet of wood. What trophies should be produced from these supplies to maximize total profit assuming that all that are made can be sold?

```
# Top Brass Optimization Model

TBmodel = Model()

#Decision Variables
@variable(TBmodel, F>=0) #football trophies
@variable(TBmodel, S>=0) #soccer trophies

#Objectives
@constraint(TBmodel, wood, 4800>= 4F+2S) #wood constraint
@constraint(TBmodel, plaques, 1750>=F+S) #Plaque constraint
@constraint(TBmodel, BS, 1500>=F) #brass soccer balls constraint
@constraint(TBmodel, BF, 1000>=S) #brass footballs constraint

#Constraints
@objective(TBmodel, Max, 9S+12F)

TBmodel
```

```
A JuMP Model
 solver: none
 objective_sense: MAX_SENSE
  objective_function_type: AffExpr
 num_variables: 2
 num_constraints: 6
  AffExpr in MOI.GreaterThan{Float64}: 4
  VariableRef in MOI.GreaterThan{Float64}: 2
Names registered in the model
 :BF, :BS, :F, :S, :plaques, :wood
```

2.5 Solving a model

After formulating a model, choose a solver that supports its mathematical structure. HiGHS handles linear, mixed-integer linear, and quadratic models. Ipopt handles smooth nonlinear models with continuous variables.

The [JuMP solver documentation](#) lists additional solvers and capabilities. Commercial solvers are not required for the standard course environment.

```
# HiGHS and Ipopt were loaded in Section 2.0.
```

There are two ways to add a solver to a JuMP model:

```
model = Model(HiGHS.Optimizer)

# ... or ...

model = Model()
set_optimizer(model, HiGHS.Optimizer)
```

A solver must support the model type. The following smooth nonlinear objective uses Ipopt; HiGHS is not an appropriate solver for this model.

```
nonlinear_model = Model(Ipopt.Optimizer)
set_silent(nonlinear_model)
@variable(nonlinear_model, 0 <= nonlinear_x <= π)
@objective(nonlinear_model, Min, cos(nonlinear_x)^2)
```

$\cos(\text{nonlinear_x})^2$

Solve the nonlinear model with Ipopt.

```
optimize!(nonlinear_model)
```

2.6 Getting solutions

Check the termination status before reading a solution. Then use `objective_value(model)` for the objective and `value(variable)` for a decision variable.

```
termination_status(nonlinear_model) == MOI.LOCALLY_SOLVED ||
    error("Ipopt stopped with status $(termination_status(nonlinear_model)).")

x_value = value(nonlinear_x)
obj_value = objective_value(nonlinear_model)

println("The optimal decision is ", x_value)
println("The objective value is ", obj_value)
```

The optimal decision is 1.5707963267948966
The objective value is 3.749399456654644e-33

More advanced: solution statuses

JuMP reports several statuses:

- `termination_status(model)` explains why the solver stopped. Common results include `OPTIMAL`, `INFEASIBLE`, `DUAL_INFEASIBLE`, and `LOCALLY_SOLVED`.
- `primal_status(model)` describes whether a primal solution is available.
- `dual_status(model)` describes whether a dual solution is available.

Only request solution values after confirming that the relevant solution exists.

```
println("Termination status: ", termination_status(nonlinear_model))
println("Primal status:      ", primal_status(nonlinear_model))
println("Dual status:       ", dual_status(nonlinear_model))
println("      x | $(x_value)")
println("       $\pi/2$  |  $$(\pi / 2)$ ")
println("-----+-----")
println("cos2(x) | $(obj_value)")
```

```
Termination status: LOCALLY_SOLVED
Primal status:      FEASIBLE_POINT
Dual status:       FEASIBLE_POINT
      x | 1.5707963267948966
       $\pi/2$  | 1.5707963267948966
-----+-----
cos2(x) | 3.749399456654644e-33
```

Before interpreting any optimization result, make status checks part of your standard workflow. Later assignments will develop the distinctions among termination, primal, and dual statuses in more detail.

Resourced Used During This Assignment

During this Assinment I used the assistance of Google Gemini via google AI search result to provide me with a quicker route to the proper julia documenation for finding the size of a matrix. I then validated this result using the link provided which lead me to the following link <https://docs.julialang.org/en/v1/base/arrays/#Base.size>